User Interfaces Design

Java - Sintassi

Giuseppe Mazzotta - Università della Calabria

Programmare in Java

Come nasce Java? Cosa serve?

- **Java Development Kit (JDK)**
  - a lezione useremo la open JDK versione 21

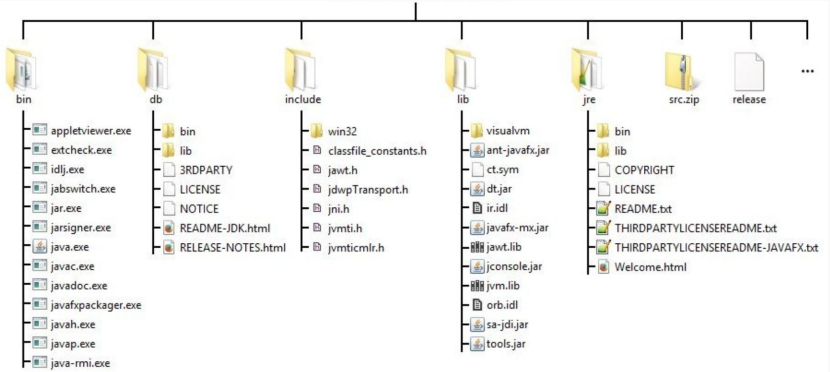
- **Un editor di testo o un IDE di sviluppo**
  - Eclipse: https://www.eclipse.org/downloads/
  - IntelliJ: https://www.jetbrains.com/idea/

Java Development Kit

La JDK contiene tutto il necessario per compilare ed eseguire i nostri programmi java

Cosa c'è al suo interno?

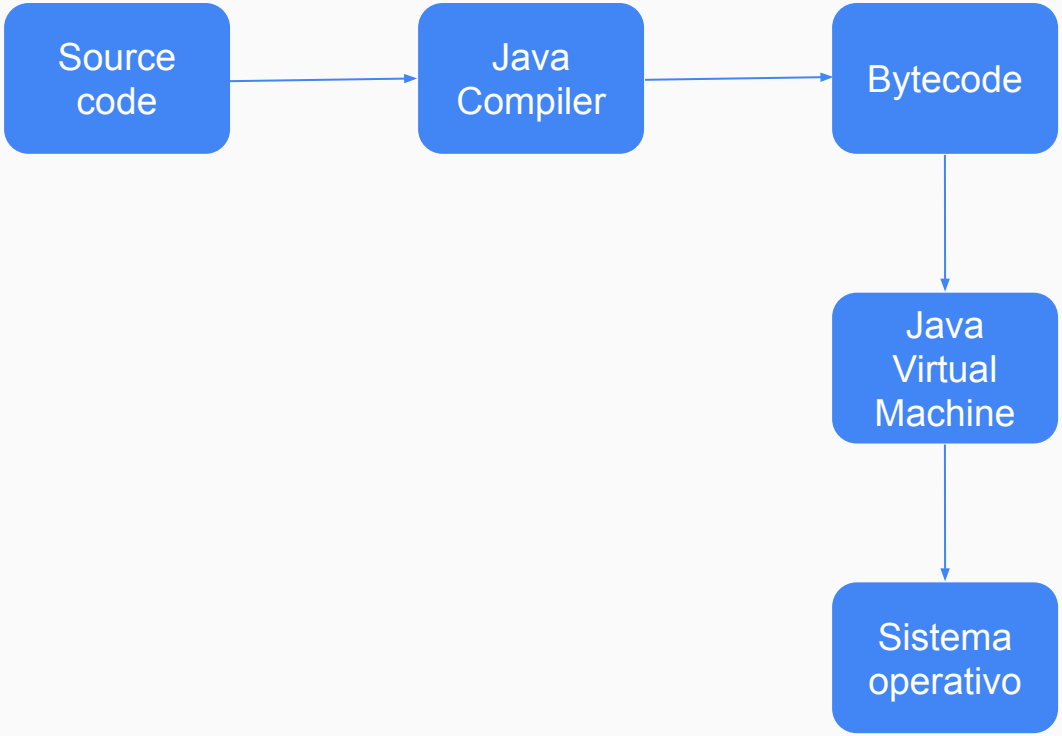
- la cartella *bin* contiene gli eseguibili dei tool di java, ad esempio il compilatore `javac`, l'interprete `java`, il disassemblatore `javap`
- la cartella *lib* contiene diverse librerie di classi usate dai vari tool
- La cartella *jre* contiene l'ambiente di esecuzione usato dalla JDK e quindi la JVM



- Codice sorgente: file .java
- Cosa contiene un file .java?
  - il file .java contiene una sola dichiarazione e definizione di classe.
- Bytecode: file .class.

  La Java virtual machine (JVM) interpreta il bytecode ed esegue il programma.

  La JVM è specifica per la piattaforma, ma l'ambiente di esecuzione è uniforme.
- Il file .java è compilato, il file .class è interpretato.



Hello World in Java

Creare il file HelloWorld.java (il nome del file è **uguale** al nome della classe)

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("hello world!"); //Stampa la stringa
    }
}
```

Compilazione	`javac HelloWorld.java`	Crea il file HelloWorld.class
Esecuzione	`java HelloWorld (linux e mac)` `java -cp . HelloWorld (windows)`	Il file .class contiene le istruzioni in un linguaggio intermedio (bytecode) che successivamente verrà interpretato

Hello World in Java

Creare il file HelloWorld.java (il nome del file è **uguale** al nome della classe)

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("hello world!"); //Stampa la stringa
    }
}
```

- Ogni file deve contenere una classe con esattamente lo stesso nome del file (anche le lettere maiuscole e minuscole)
- A differenza del C++, la keyword `public`, `private` o `protected` va inserita prima di ogni campo, metodo o classe
- Il main riceve un array di stringhe che rappresentano i parametri da linea di comando del programma

Parametri da linea di comando

```
public static void main(String[] args)
```

```
> java AnApplication ciao mondo
```

Due argomenti, `ciao` e `mondo`, sono caricati nell'array args

```
> java AnApplication
```

Nessun argomento da linea di comando (l'array args è vuoto)

- Ogni stringa di caratteri separata da spazio che segue l'invocazione di AnApplication è un parametro da linea di comando.
- Ogni parametro viene messo a disposizione come elemento dell'array args (nome arbitrario) passato al main.
- Se args è vuoto, nessun parametro è stato fornito.

Naming convention

Per naming convention si intende la regola di scrittura da utilizzare per la scelta dei nomi all'interno del programma.

- I nomi delle classi iniziano per lettera maiuscola. Nel caso in cui il nome della classe sia composto da più parole si deve utilizzare la lettera maiuscola per ogni parola. Ad esempio: `class DatabaseManager`.
- I nomi delle variabili e dei metodi iniziano per lettera minuscola. Nel caso in cui il nome sia composto da più parole si deve utilizzare la lettera maiuscola per le parole successive alla prima. Ad esempio: `int maxSize`.

Tipi base

In Java abbiamo 8 tipi base:

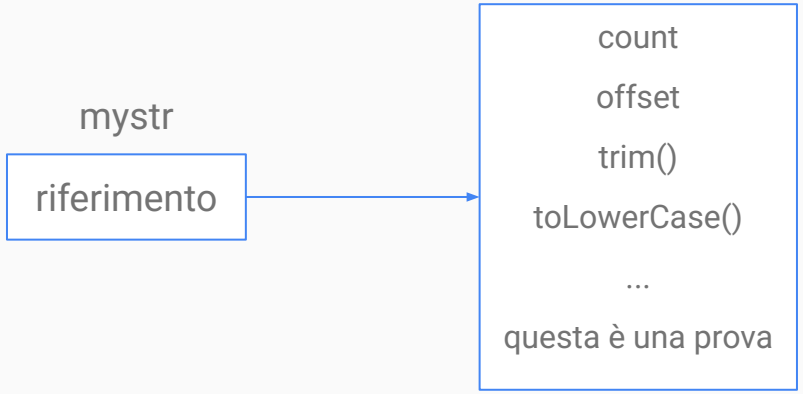
1. byte
2. boolean
3. short
4. int
5. long
6. float
7. double
8. char

Cosa succede quando creiamo un oggetto?

```
String mystr = "questa è una prova";
```

Cosa sono i riferimenti?

Concettualmente sono simili ai puntatori del C++: un riferimento rappresenta un indirizzo di memoria dove è stato allocato un oggetto ma a differenza del puntatore non è un indirizzo di memoria.



Importante:

- Non possono essere modificati o inizializzati con valori arbitrati
- non vanno deallocati! C'è il Gargabe Collector

I riferimenti

```
String mystr = "questa è una prova";
```

Qual è il riferimento di `mystr`?

String@6d06d69c

A sinistra del simbolo @ troviamo il tipo dell'oggetto, nel nostro caso String.
A destra del simbolo @ troviamo un valore esadecimale che rappresenta l'indirizzo effettivo dell'oggetto.

Se si prova a stampare `mystr`, si ottiene in output il riferimento di `mystr`.

Array

Array di tipo int e di lunghezza `n`: `int[] a = new int[n];`

Un array può anche essere creato assegnando dei valori: `int[] a = {6,2,4,8};`

Per conoscere la size di un array si può utilizzare il campo `length`. Nell'esempio precedente, `a.length` è uguale a 4.

Come in C++, gli indici partono da 0 e arrivano alla length-1. Si può accedere ad una posizione usando le parentesi quadre: `a[2] = 5;`

Matrici

	0	1	2
0	1	2	3
1	4	5	6

Matrice di tipo int con `r` righe e `c` colonne: `int[][] m = new int[r][c];`

Assegnando dei valori: `int[][] m = {{1,2,3}, {4,5,6}};`

Per conoscere il numero di righe di una matrice si può utilizzare `length`.
Nell'esempio precedente, `m.length` è uguale a 2, mentre `m[0].length` è pari a 3.

È possibile creare matrici dove ogni riga ha una dimensione diversa.

```
int[][] m = new int[2][]; m[0] = new int[3]; m[1] = new int[4];
```

Si può accedere ad una posizione usando le parentesi quadre, `m[0][1] = 7;`

Sintassi di un programma Java

- Molto simile alla sintassi del C++.
- Si possono usare tutti i costrutti visti per C++: if, for, while, do while, switch, ecc.
- Ogni file che si crea deve contenere una classe con lo stesso nome (nell'esempio Main.java).

```java
public class Main {
    public static void main(String[] args) {
        int a, b = 1;
        a = 0;
        int[] array = new int[10];
        if(a+b >= 1) {
            for(int i = 0; i < array.length; i++)
                array[i] = 1;
        }
        else {
            int i = 0;
            while(i < array.length) {
                array[i] = 0;
                i++;
            }
        }
        switch(array[0]) {
            case 0:
                //do something
                break;
            case 1:
                //do something
                break;
            default:
                //do something
                break;
        }

        for(int v : array) //for each
            System.out.println(v);
    }
}
```

Sintassi di un programma Java

Nelle nuove versioni di Java, ci sono anche dei costrutti che permettono di scrivere diversamente il codice

```java
public class Main {
    public static void main(String[] args) {
        var a = 1; //il tipo di a è inferito automaticamente
        int[] array = new int[10];
        for(int i = 0; i < array.length; i++)
            array[i] = i;


        switch(array[0]) {
            case 0 -> System.out.println("Contiene " + 0);
            case 1 -> System.out.println("Contiene " + 1);
            default -> System.out.println("Contiene altri valori");
        }

        //Possiamo fare in modo che lo switch restituisca qualcosa
        //res conterrà 0 se array[0] è 0
        //res conterrà 1 se array[0] è 1
        var res = switch(array[0]) {
            case 0 -> { System.out.println(0); yield 0; }
            case 1 -> { System.out.println(1); yield 1; }
            default -> { yield "Numero non supportato"; }
        };
        //res è dichiarato come var, quindi il tipo è inferito
        //sarà int nel caso di 0 e 1 o una stringa negli altri casi
    }
}
```

Classi

```
public class Esempio {
  public Esempio() {}

  public Esempio(int c) { campo = c; }

  public int getCampo() { return this.campo; }

  public void setCampo(int c) { this.campo = c; }

  int campo;
  public int campoPubblico;
  private int campoPrivato;
  protected int campoProtected;
}
```

Ogni file .java contiene, solitamente, la dichiarazione e la definizione di una sola classe.

La classe si dichiara con l'istruzione:

```
public class NomeClasse
```

Gli specificatori di accesso (`public`, `private`, `protected`) vanno inseriti prima della dichiarazione del campo o del metodo.

Se un campo/metodo non ha uno specificatore di accesso, sarà visibile nella classe e nelle classi appartenenti allo stesso package.

Creazione oggetto:

```
Esempio e1 = new Esempio();
Esempio e2 = new Esempio(4);
```

Esempio

```
public class Main {
  public static void main(String[] args) {
    Esempio e1 = new Esempio(1);
    Esempio e2 = new Esempio(2);
    System.out.println(e1.getCampo()); //Risultato: 1
    System.out.println(e2.getCampo()); //Risultato: 2
    e2 = e1;
    System.out.println(e2.getCampo()); //Risultato: 1

    e2.setCampo(3);
    System.out.println(e2.getCampo()); //Risultato: 3
    System.out.println(e1.getCampo()); //Risultato: 3

    e1.setCampo(5);
    System.out.println(e1.getCampo()); //Risultato: 5
    System.out.println(e2.getCampo()); //Risultato: 5
  }
}
```

Esempio

```java
public class Esempio {
  private int campo;
  public Esempio(int c) { campo = c; }
  public int getCampo() { return campo; }
  public void modifica(Esempio e1, Esempio e2) {
    campo = e1.campo + e2.campo;
    e1.campo=0;
    e2.campo=0;
    e1 = new Esempio(8); //Ha effetto solo nel metodo
    System.out.println(e1.getCampo()); //Risultato: 8
  }
  public static void main(String[] args) {
    Esempio e1 = new Esempio(1);
    Esempio e2 = new Esempio(2);
    Esempio e3 = new Esempio(5);
    System.out.println(e3.getCampo()); //Risultato: 5

    e3.modifica(e1,e2);
    System.out.println(e1.getCampo()); //Risultato: 0
    System.out.println(e2.getCampo()); //Risultato: 0
    System.out.println(e3.getCampo()); //Risultato: 3
  }
}
```

I METODI IN JAVA

- Tutti gli argomenti ai metodi sono passati come una copia del valore.
- Se l'argomento è un tipo base (int, float, ecc.), allora viene fatta una copia dell'elemento.
- Se l'argomento è una variabile riferimento, allora viene fatta una copia del suo riferimento.

Esempio

```
public class Sommatore {
  public void somma(int… c) {
    int res = 0;
    for(int v: c)
      res += v;
    System.out.println("Sum: "+res);
  }
  public void somma(char… c) {
    int res = 0;
    for(char v: c)
      if(v >= '0' && v <= '9')
        res += v - '0';
    System.out.println("Sum: "+res);
  }
  public static void main(String[] args) {
    Sommatore s = new Sommatore();
    s.somma(1,2,3);                  //Risultato: 6
    s.somma('1','a','2','b','3'));  //Risultato: 6
  }
}
```

- Un metodo può avere un numero indefinito di parametri dello stesso tipo ma a patto che sia l'ultimo.
- La segnatura di un metodo è data dal suo nome, dai tipi dei parametri e dal loro ordine (il tipo di ritorno **NON** fa parte della segnatura).
- **Overloading:** posso avere metodi con lo stesso nome ma ordine o tipi di parametri diversi.
- **Vantaggio**: non devo cercare nomi diversi per metodi che implementano una logica simile
- Esempio molto comune: i costruttori

La keyword static

```
public class MyMath {
  public static int gcd(int m, int n) {
    if (n == 0)
      return m;
    else
      return gcd(n, m % n);
  }
  public static void main(String[] args) {
    int result = MyMath.gcd(60,24);
    System.out.println(result);
  }
}
```

- Davanti a un membro di una classe (campo/metodo) si può usare la keyword static
- I membri static esistono indipendentemente dagli oggetti istanziati:
  - si può accedere ad essi tramite il nome della classe.
- I campi static sono condivisi da tutte le istanze degli oggetti.
- I metodi static non possono utilizzare campi o invocare metodi che non siano static.

Package

- I package sono il meccanismo attraverso il quale si possono creare librerie di classi correlate (come le librerie C++). Esempi di package disponibili nella JDK `java.lang`, `java.net` e molti altri.

- Perchè sono importanti i package?
  - Permettono di raggruppare i tipi secondo criteri funzionali e di correlazione.
  - Riusare classi scritte da altri programmatori importando i package in cui esse sono definite.
  - Fornire un ulteriore livello di accesso (lo specificatore di visibilità di default consente l'accesso ai membri interni alle classi nello stesso package).
  - Permette di disambiguare classi aventi lo stesso nome (**NOTA** una classe è identificate univocamente dal package e dal nome classe).

Package

- Tutte le classi facenti parte di un package iniziano con la direttiva `package <nome>`, dove `<nome>` è l'identificativo del package.

- Per utilizzare le classi di un package all'interno di altre classi, è necessario specificare direttive import (simile a include del c++) nell'intestazione delle classi utilizzatrici.

- Se usiamo una classe che è all'interno dello stesso package non è necessario importarla.

- È possibile che i package contengano sotto-package.

  Esempi di import:
  `import java.util.Scanner` importa la classe `Scanner`, del package `java.util` (sottopackage del package `java`).
  `import java.io.*` importa tutte le classi e gli eventuali sotto-package di `java.io`.

Moduli

- A partire da Java 9 è stato introdotto un nuovo livello di astrazione superiore ai package, conosciuto come Java Platform Module System (JPMS), o semplicemente Moduli.
- I moduli sono gruppi di package legati tra di loro e gruppi di risorse. Orientativamente è come se fossero **“package di package”**.
- Ogni modulo è responsabile delle proprie risorse, come ad esempio immagini, eventuali file, ecc.
- Per utilizzare i moduli si deve realizzare un file descrittivo che definisce diversi aspetti del modulo:
  - il nome del modulo
  - le dipendenze (altri moduli da cui il modulo dipende)
  - una lista di package pubblici (cioè, package che si vuole rendere visibili da altri moduli)
  - ecc.
- I package all’interno di un modulo sono di default privati, questo significa che devono essere resi pubblici in modo esplicito se si vuole renderli disponibili al di fuori del modulo che si sta creando.

Input da console

```
import java.util.Scanner;

public class EsempioLetturaInput {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    System.out.println("Primo numero");
    int m = in.nextInt();
    System.out.println("Secondo numero");
    int n = in.nextInt();
    int z = m + n;
    System.out.println("Somma: " + z);
  }
}
```

- La classe `Scanner` di `java.util` mette a disposizione metodi per iterare oggetti di tipo `InputStream` tra cui anche `System.in`.
- Esiste un metodo `next<...>` per quasi tutti i tipi primitivi, ad esempio `nextInt`, `nextLong`, ecc.
- Per le stringhe ci sono i metodi `next` (prossima stringa) e `nextLine` (prossima riga).

Oggetti speciali: stringhe

- Gli oggetti `String` rappresentano sequenze di caratteri alfanumerici e sono immutabili.
- È possibile accedere ad un qualsiasi carattere con il metodo `charAt`, ma non è possibile modificarlo.
- Data una stringa `str`, gli indici dei caratteri vanno da `0` a `str.length() - 1`.
- Esistono metodi per l'uguaglianza e per la concatenazione. Ad esempio, l'istruzione

  ```
  String c = a + b
  ```

  crea una nuova stringa con i contenuti di `a` concatenati a quelli di `b`.

Confronto stringhe

```java
import java.util.Scanner;

public class TestString {
  public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    String text1 = in.nextLine();
    String text2 = in.nextLine();
    //Confronto del riferimento
    if(text1 == text2)
      System.out.println("Confronto ==");

    //Confronto del valore
    if(text1.equals(text2))
      System.out.println("Confronto equals");

    //Confronta ignorando maiuscole/minuscole
    if(text1.equalsIgnoreCase(text2))
      System.out.println("Confronto equalsIgnoreCase");
  }
}
```

Stringhe formattate

```
public class TestString {
  public static void main(String[] args) {
    String text = "ciao\n\tcome va?\n\tTutto ok\n" ;
    String formattedText = """
ciao
    come va?
    Tutto ok

""";
    System.out.println(text);
    System.out.println(formattedText);
  }
}
```

StringBuffer e StringBuilder

```
String x = "Testo"; // Un oggetto String
String y = x + "!"; // Un nuovo oggetto String

StringBuffer z = new StringBuffer(x);
z.append("!");
z.setCharAt(0, 't');

StringBuilder w = new StringBuilder(x);
w.append("!") ;
w.setCharAt(0, 'u');
```

- La classe `String` costruisce oggetti immutabili.
- Ogni operazione su oggetti `String` che ha come risultato `String` genera nuovi oggetti.
- Le classi `StringBuffer` e `StringBuilder` mettono a disposizione oggetti mutabili per la gestione di testi.
- `StringBuilder` è tipicamente più efficiente di `StringBuffer` ma non garantisce la sincronizzazione nella gestione multi-thread.

Differenze di performance (1)

```
import java.util.Random;

public class Main {
    private static String stampaConString() {
        String s = "";
        for(int i = 0; i < 100000; i++)
            s += i;
        return s;
    }

    //In questo caso è più veloce
    private static String stampaConStringBuilder() {
        StringBuilder s = new StringBuilder();
        for(int i = 0; i < 100000; i++)
            s.append(i);
        return s.toString();
    }
```

Differenze di performance (2)

```
final public static int  value = 20000;

//In questo caso è più veloce
private static void testConString(int n) {
    for (int i = 0; i < value; i++)
        for (int j = 0; j < value; j++) {
            String str =  "Prova " + i + " " + j;
            if(i == j && i == n) { System.out.println(str); }
        }
}

private static void testConStringBuilder(int n) {
    for (int i = 0; i < value; i++)
        for (int j = 0; j < value; j++) {
            StringBuilder s =  new StringBuilder();
            s.append("Prova "); s.append(i); s.append(" "); s.append(j);
            String str = s.toString();
            if(i == j && i == n) { System.out.println(str); }
        }
}
```

Differenze di performance (3)

```
public static void test() {
    long startTime = System.nanoTime();
    stampaConString();
    long stopTime = System.nanoTime();
    double duration = (double) (stopTime - startTime) / 1000000000;
    System.out.println("Durata primo metodo: " + duration + " secondi");
    startTime = System.nanoTime();
    stampaConStringBuilder();
    stopTime = System.nanoTime();
    duration = (double) (stopTime - startTime) / 1000000000;
    System.out.println("Durata secondo metodo: " + duration + " secondi");
}
```

Differenze di performance (4)

```
    public static void test2() {
        Random random = new Random();
        int v = random.nextInt(value);
        long startTime = System.nanoTime();
        testConString(v);
        long stopTime = System.nanoTime();
        double duration = (double) (stopTime - startTime) / 1000000000;
        System.out.println("Durata primo metodo: " + duration + " secondi");
        startTime = System.nanoTime();
        testConStringBuilder(v);
        stopTime = System.nanoTime();
        duration = (double) (stopTime - startTime) / 1000000000;
        System.out.println("Durata secondo metodo: " + duration + " secondi");
    }

    public static void main(String[] args) {
        test();
        test2();
    }
}
```

Record

È comune realizzare delle classi che semplicemente si occupano di gestire dei dati, ad esempio nel caso di risultati di una query di un database o per prendere informazioni da un servizio online, ecc.

Tipicamente queste classi sono immutabili, perché questo garantisce alcune invarianti sul codice, ad esempio le rende thread-safe.

Per implementare questo tipo di classi di solito si deve:

- rendere `private` e `final` ogni attributo della classe.
- implementare solo i getter per gli attributi della classe.
- implementare un costruttore pubblico con un numero di parametri pari agli attributi della classe.
- implementare il metodo `equals`.
- implementare il metodo `hashCode`.
- implementare il metodo `toString`.

Gli ultimi 3 metodi li vedremo più avanti.

Java mette a disposizione i record per creare in modo automatico questo tipo di classi.

Record

La definizione di un record è molto semplice:

```
public record Person(String firstName, String lastName) {
}
```

Il suo utilizzo è poi simile a quello di ogni altro oggetto:

```
//Creazione dell'oggetto con il costruttore
Person p = new Person("Mario", "Rossi");
//Viene generato un metodo get per ogni campo privato
//Il metodo ha lo stesso nome dell'attributo
System.out.println(p.firstName());
System.out.println(p.lastName());
```

Record

```
public record Person(String firstName, String lastName) {

    //Si può definire un costruttore con un numero di parametri diversi
    public Person(String firstName) {
        //Deve invocare il costruttore con due parametri
        this(firstName, "default value");
    }
    //Si può ridefinire il costruttore per effettuare dei controlli
    public Person(String firstName, String lastName) {
        Objects.requireNonNull(firstName, "first name cannot be null");
        Objects.requireNonNull(lastName,  "last name cannot be null");
        this.firstName = firstName;
        this.lastName = lastName;
    }
    //Può contenere campi e metodi statici
    public static final String NONE=”none”;
    public static Person getDummy(){ return Person(NONE,NONE);}
}
```